Centro Universitário de Excelência
Sistemas de Informação

Refatoração e Evolução da API DendêEventos

Autor: André Almeida Gomes Neto
Augusto Brandão Ferreira
Deivid Souza dos Santos Oliveira
Hellen Verena da Conceição Magalhães
Ismael Rodrigues de Oliveira Neto
João Vítor da Conceição de Almeida

Feira de Santana, 2026

Agenda

Refatoração para tornar a API mais organizada, modular e escalável.

1. Introdução:
Objetivo e visão geral da refatoração

2.Diagrama:
Diagrama de Classe, Diagrama de Atividade

3. Builder:
Uso do padrão de projeto Builder na classe Evento.

4. Dtos:
Criação de Data Transfer Objects para requisições e respostas

5. Controllers:
Construção de Controllers para lidar com as Rotas da Api

6. Mappers:
Mapeamento entre o Modelo e os Dtos.

7. Considerações Finais:
Desafios enfrentados

8. Referências.

Introdução

Dendê Eventos API

Refatoração para tornar o sistema mais organizado, modular e fácil de evoluir.

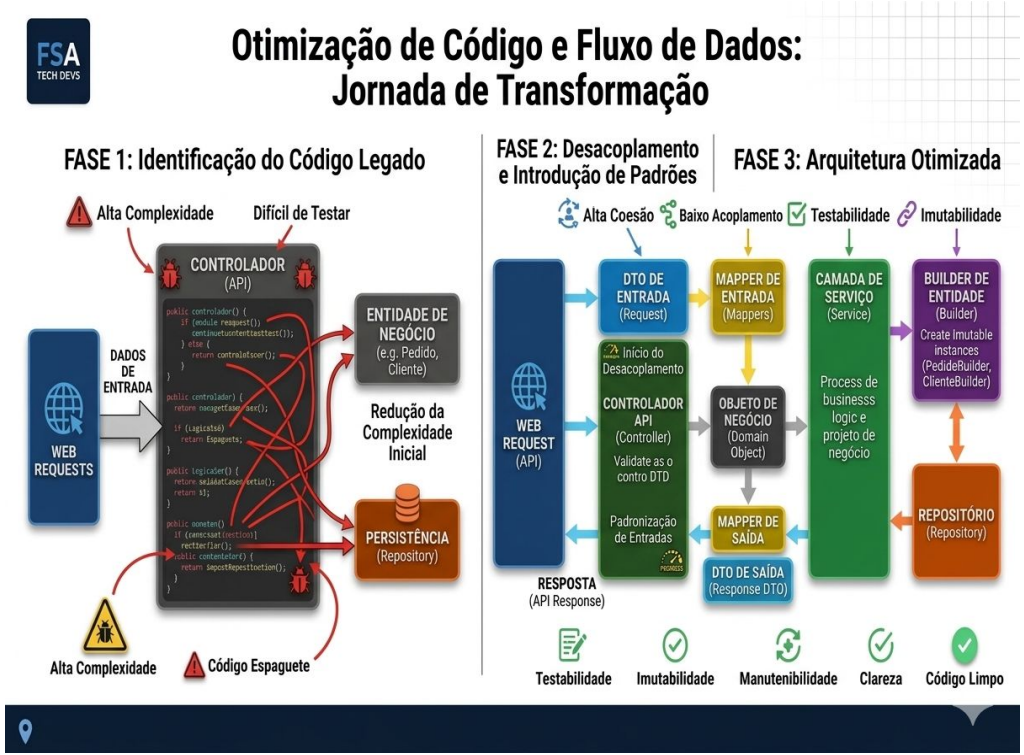


Diagrama de Classe

Diagrama de Classe

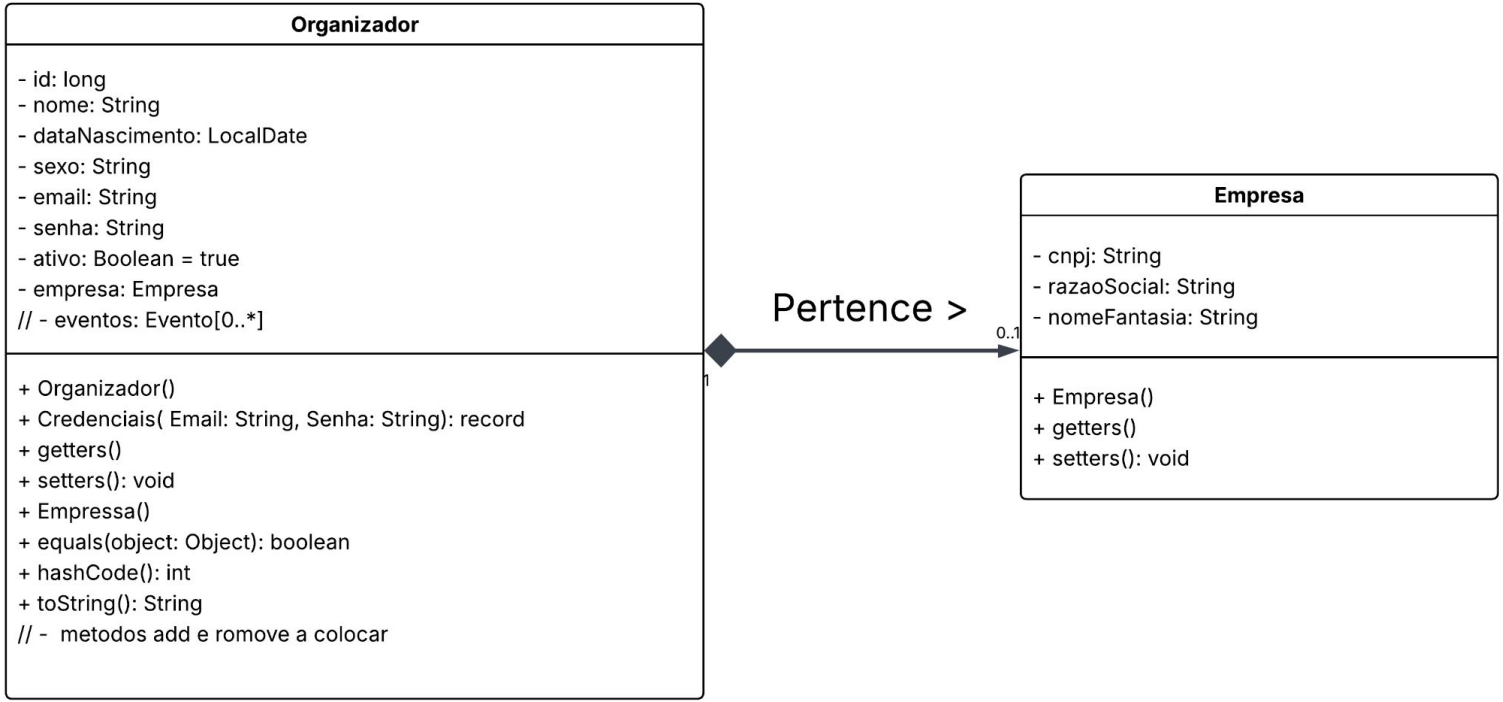


Diagrama de Classe

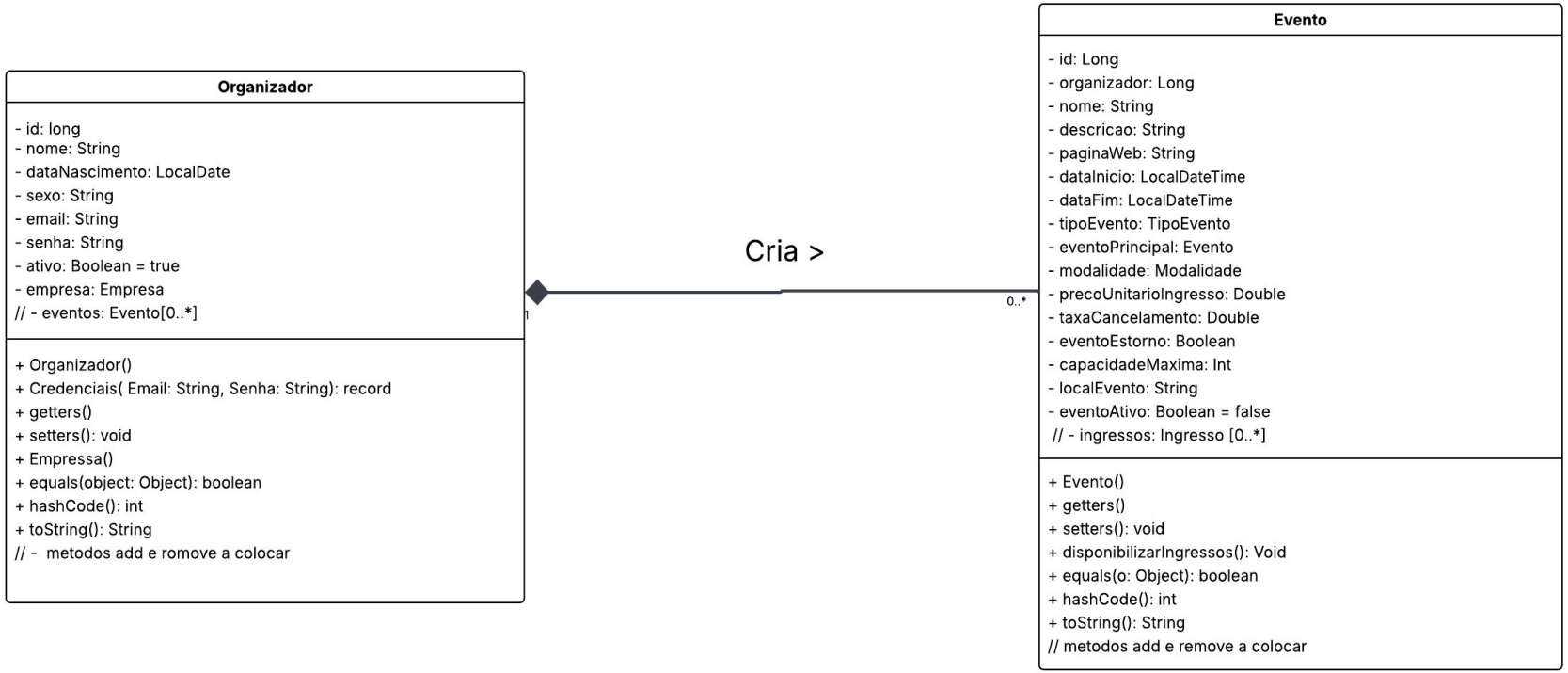


Diagrama de Classe

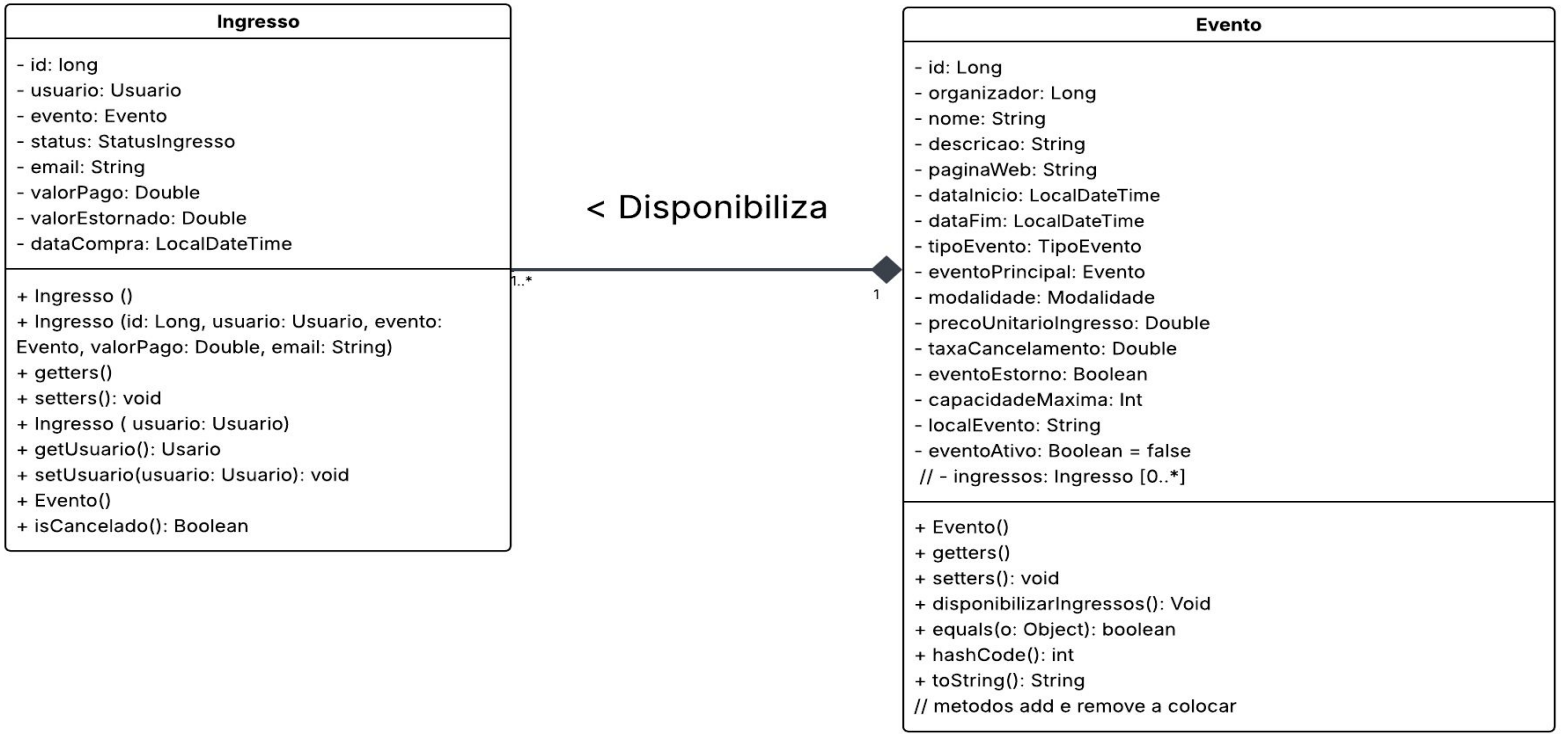


Diagrama de Classe

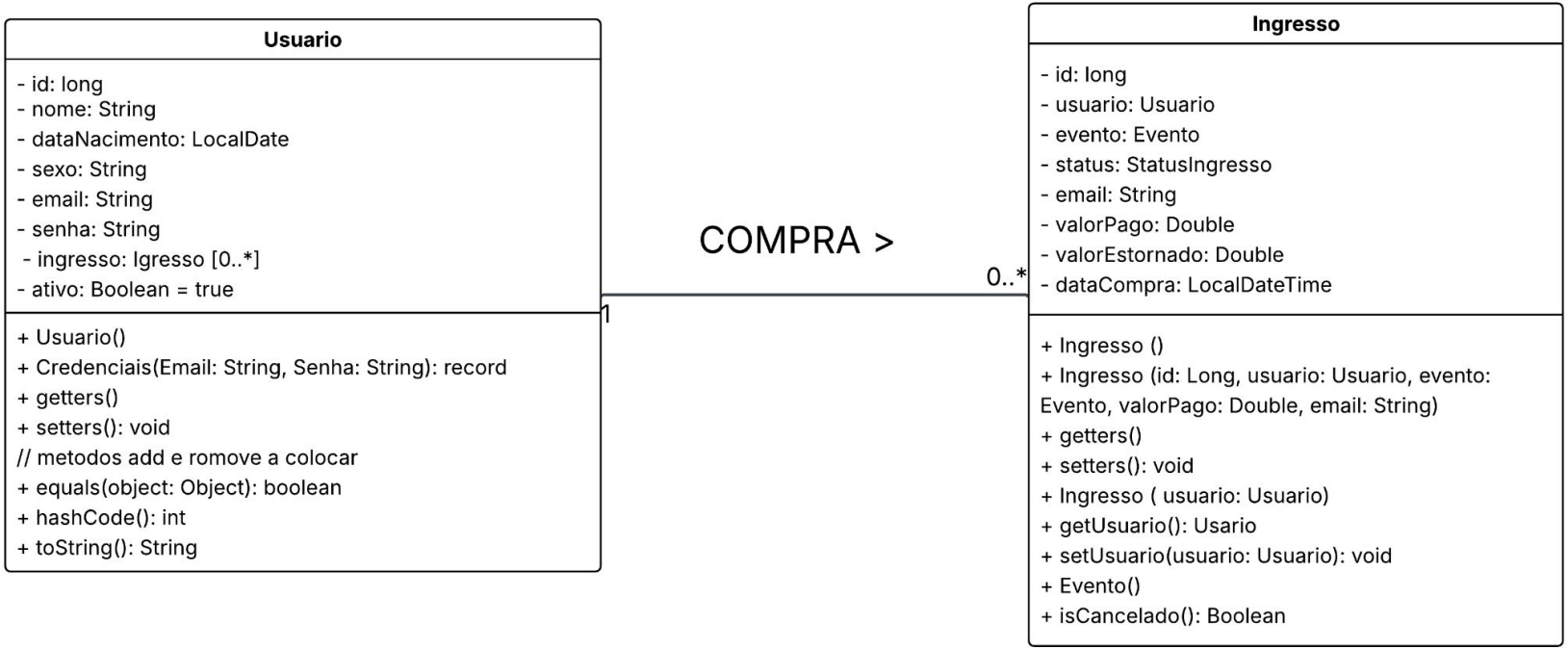


Diagrama de Atividade

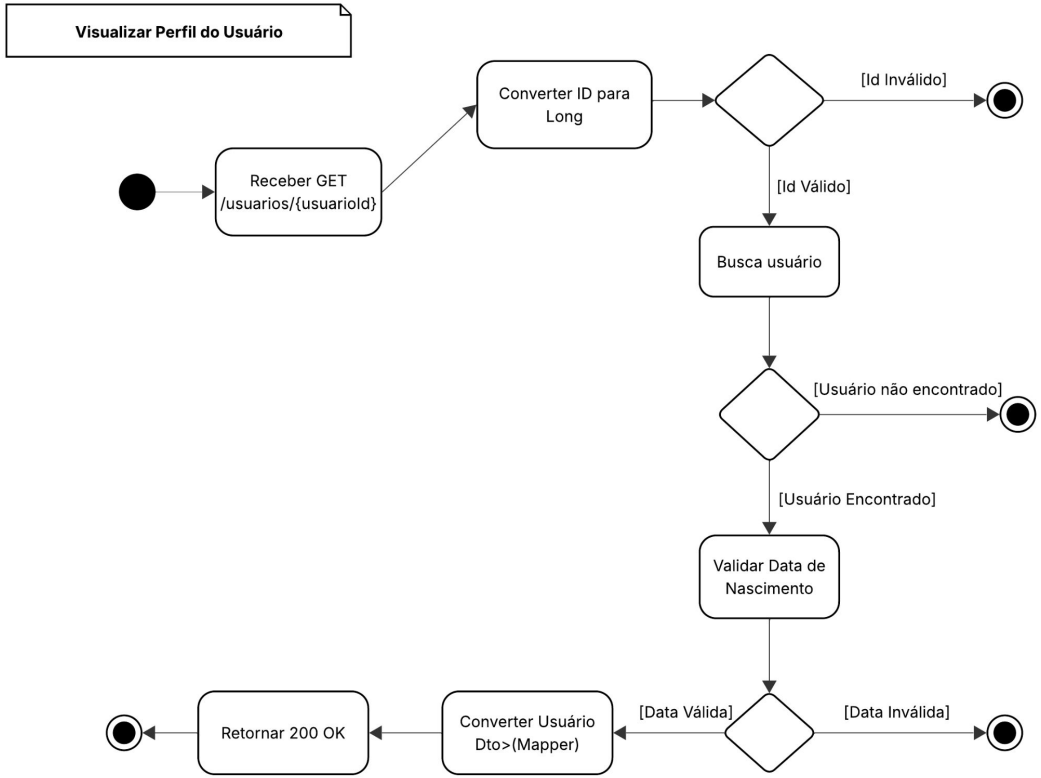
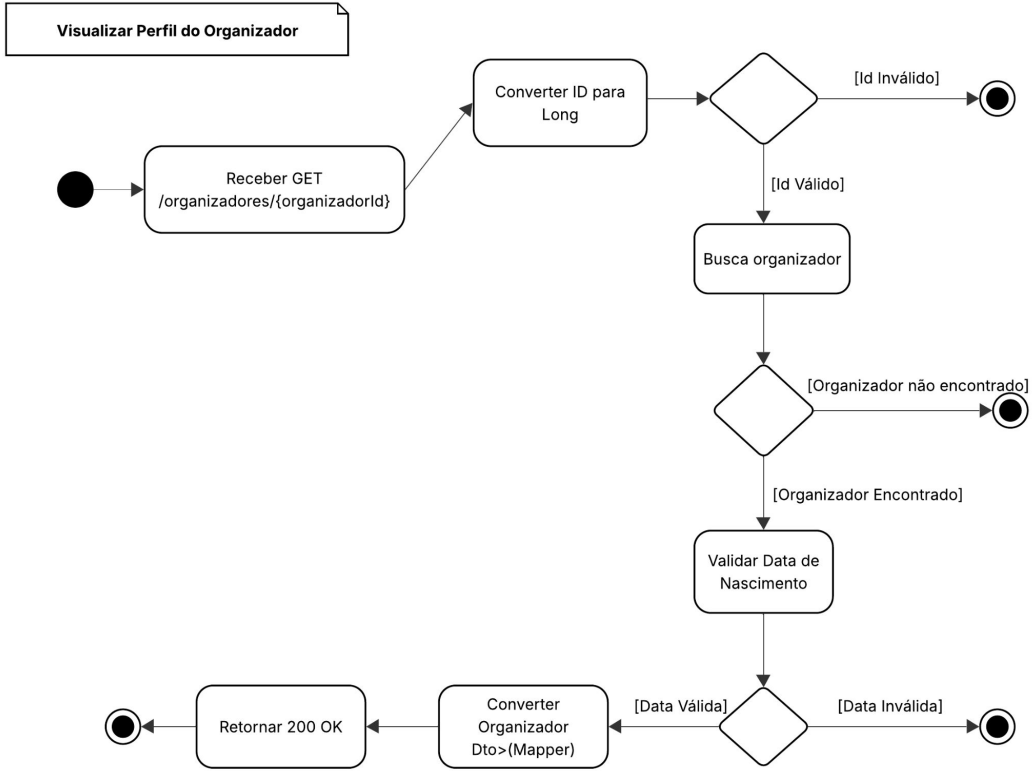
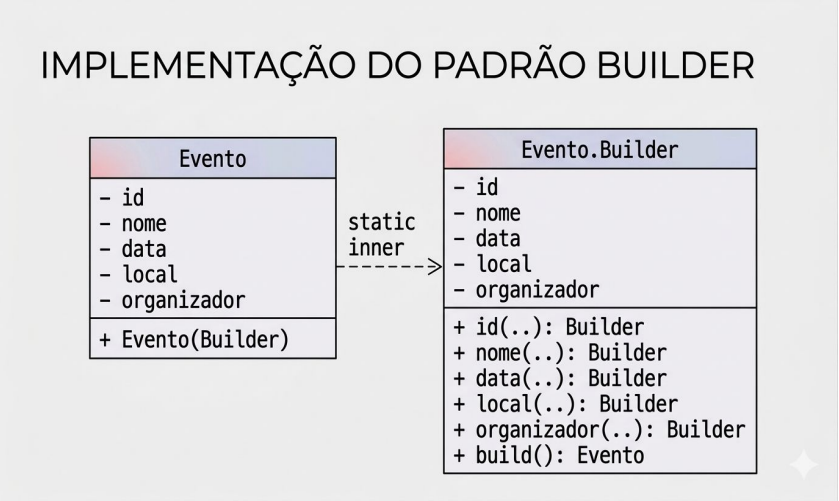


Diagrama de Atividade



Builder

- Builder é um padrão de projeto criacional
- Utilizado para criar objetos com muitos parâmetros
- Permite construção passo a passo



O Builder é implementado como uma classe interna (Evento.Builder) que define os atributos e fornece métodos encadeados (ex: nome(), data()). Ao final, o método build() cria e retorna o objeto Evento.

Data Transfer Object (DTO)

- DTOs transportam dados entre cliente e API
- Permitem controlar o que é enviado e recebido

Problemas:

- Exposição direta do Model
- Risco de vazar dados sensíveis
- Alto acoplamento

DTOs para Eventos - Aplicação Prática

Ação da API	DTO Utilizado	Por que DTO?
Atualizar Evento	AtualizarEventoDto	Controlar campos editáveis.
Cadastrar Evento	CadastrarEventoDto	Validar e garantir integridade.
Listar Eventos	ListarEventoDto	Otimizar e filtrar dados.
Feed Principal	FeedEventoDto	DTO leve para resumo.
Atualizar Status	StatusEventoDto	Operação atômica e segura.

Data Transfer Object (DTO)

DTOs do Usuário:

Em verde : DTOs de Saída

Em Azul: DTOs de Entrada

CadastrarUsuarioDTO
- nome: String
- dataNascimento: LocalDate
- sexo: String
- email: String
- senha: String

AtualizarUsuarioDTO
- nome: String
- dataNascimento: LocalDate
- sexo: String
- senha: String

ListaIngressosUsuarioDto
- usuarioId: Long
- ingressoId: Long
- eventoId: Long
- eventoNome: String
- dataInicio: LocalDateTime
- status: String
- valorPago: Double
- valorEstornado: Double
- eventoAtivo: boolean
- dataCompra: LocalDateTime
- email: String

VisualizarUsuarioDTO
- nome: String
- dataNascimento: LocalDate
- idade: String
- sexo: String
- email: String
- ativo: Boolena

StatusUsuarioDTO
- mensagem: String
- usuarioId: Long
- ativo: Boolean

CancelarIngressoUsuarioDTO
- mensagem: String
- ingressoId: Long
- valorEstornado: Double

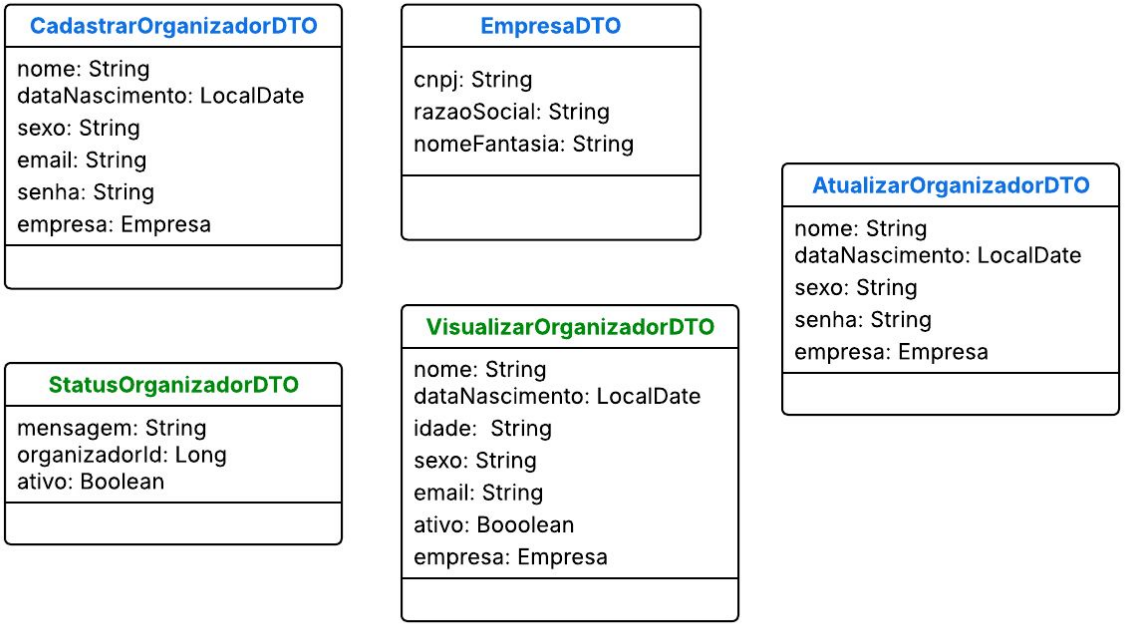
ComprarIngressoUsuarioDTO
- usuarioId: Long

Data Transfer Object (DTO)

DTOs do Organizador:

Em verde : DTOs de Saída

Em Azul: DTOs de Entrada



Data Transfer Object (DTO)

DTOs do Evento:

Em verde : DTOs de Saída

Em Azul: DTOs de Entrada

AtualizarEventoDTO

nome: String
descricao: String
paginaWed: String
dataInicio: LocalDateTime
dataFim: LocalDateTime
tipoEvento: String
eventoPrincipalId: Long
modalidade: String
precoUnitarioIngresso: Double
taxaCancelamento: Double
eventoEstorno: Boolean
capacidaeMaxima: Integer
localEvento: String

CadastrarEventoDTO

nome: String
descricao: String
paginaWeb: String
dataInicio: LocalDateTime
dataFim: LocalDateTime
tipoEvento: String
eventoPrincipalId: Long
modalidade: String
precoUnitarioIngresso: Double
taxaCancelamento: Double
eventoEstorno: Boolean
capacidadeMaxima: Integer
localEvento: String

EventosOrganizadorDTO

nome: String
dataInicio: LocalDateTime
dataFim: LocalDateTime
local: String
precoIngresso: Double
capacidadeMaxima: Integer

StatusEventoDTO

mensagem: String
eventoId: Long
ativo: Boolean

FeedEventoDTO

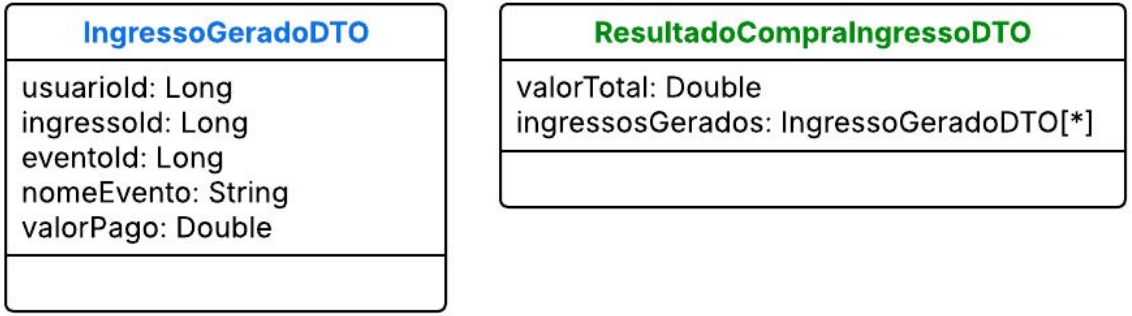
nome: String
dataInicio: LocalDateTime
dataFim: LocalDateTime
local: String
precoIngresso: Double
capacidadeMaxima: Integer
tipoEvento: String
modalidade: String

Data Transfer Object (DTO)

DTOs do Ingresso:

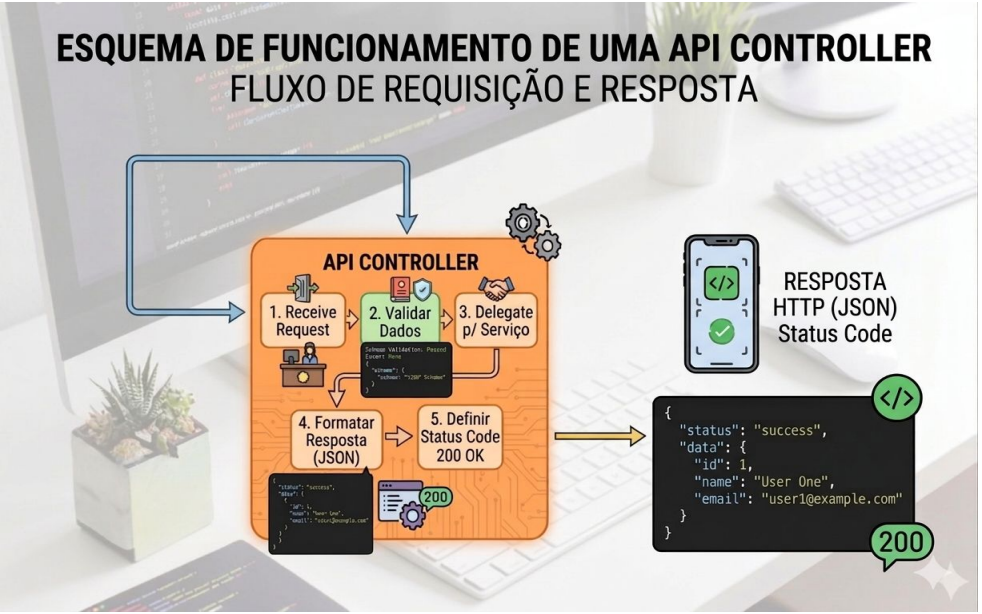
Em verde : DTOs de Saída

Em Azul: DTOs de Entrada



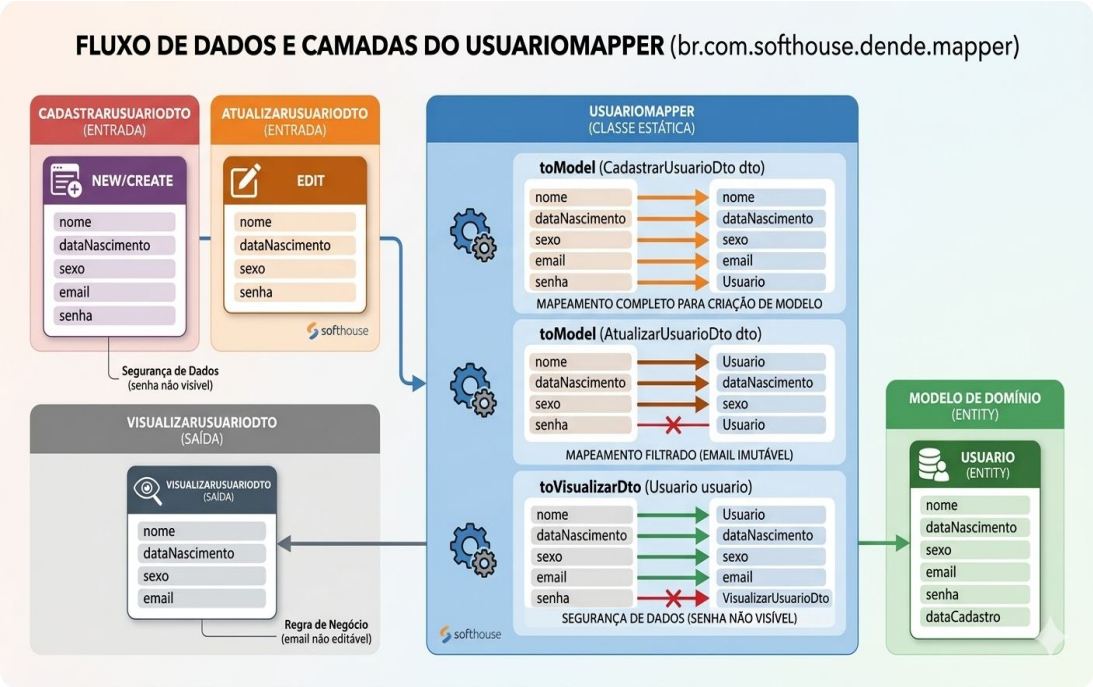
Controllers

- Controllers organizados por entidade
- Cada camada tem uma responsabilidade
- Controller → recebe e responde
- Service → regra de negócio
- Repository → acesso a dados

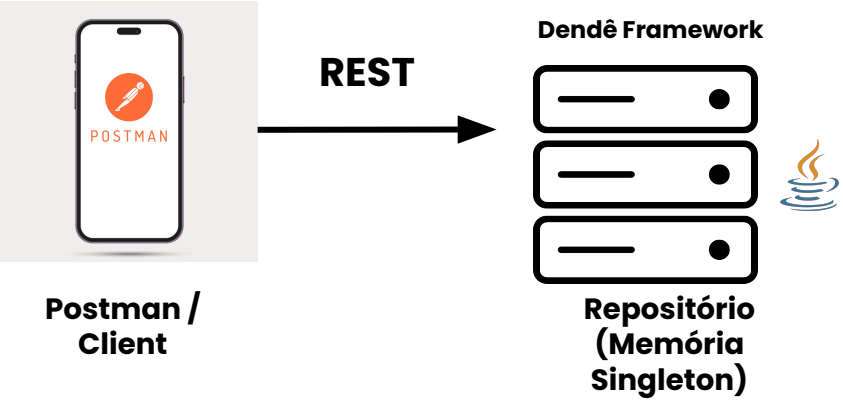


Mappers

- Convertem Model ↔ DTO
- Organizam o fluxo de dados entre as camadas
- Objetivo: reduzir acoplamento e evitar exposição direta das entidades.
- Organiza e desacopla código entre Modelo e Dtos.



Metodologia: Ferramentas



Controle de Versão e Repositório

Resultados

Builder Classe eventos

Implementação do Builder para criação de Eventos

- Uso do Builder Pattern na classe Evento
- Criação de objetos mais organizada e legível
- Evita construtores com muitos parâmetros
- Mais flexibilidade na criação de eventos

Comparativo de Implementação do Padrão Builder

SEM BUILDER (Construtor Completo)

```
// Código sem padrão Builder: construtor complexo
Evento e = new Event((
    1,                      // id
    "Grande Show",          // nome
    "Estádio Nacional",     // local
    new Date(),             // data
    10000,                  // vagas
    150.00,                 // preço
    true,                   // ativo
    "Música",               // categoria
    "Um show imperdível!"   // descrição (opcional)
);
```

Ordem crítica dos parâmetros. Difícil de ler e manter.

COM BUILDER (Padrão Builder Implementado)

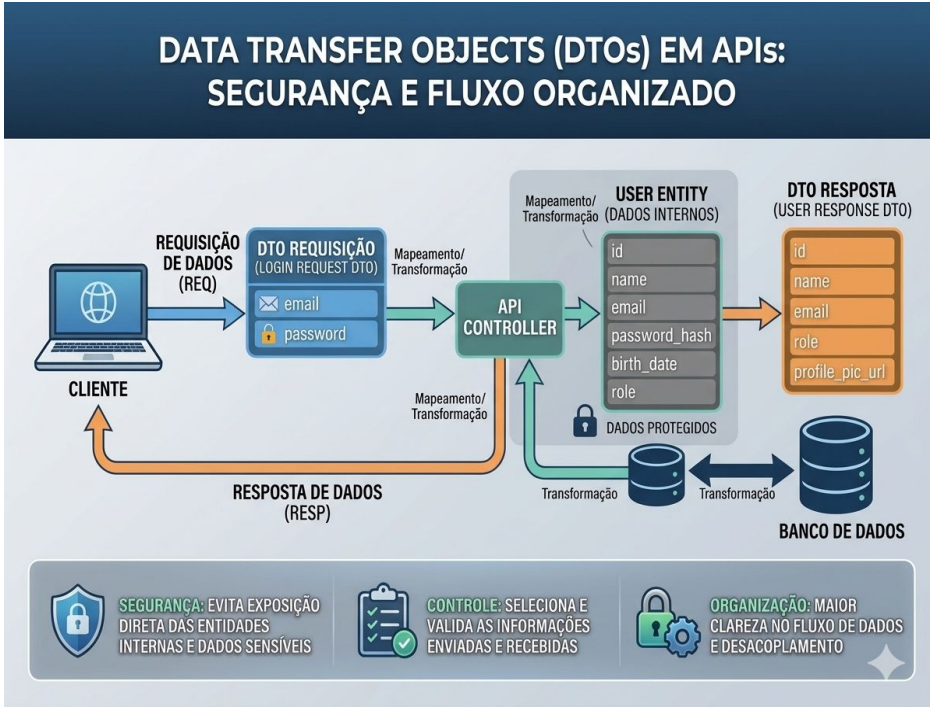
```
// Código com padrão Builder (Descrição e categoria om
Evento e = new Evento.Builder()
    .id(1)                      // id
    .nome("Grande Show")        // nome
    .local("Estádio Nacional") // local
    .data(new Date())           // data
    .vagas(10000)               // vagas
    .preco(150.00)              // preço
    .ativo(true)                // ativo
    // Categoria e descrição não definidos (usam padrã
    .build();
```

Clareza nos nomes dos métodos. Permite omitir parâmetros opcionais.

Resultados

DTOS

- Uso de DTOs para requisições e respostas.
- Controle dos dados da API
- Evita exposição direta das entidades.
- Mais segurança e organização.



Resultados

Controllers

Estruturação dos Controllers

- Controller: recebe requisições, valida e responde.
- Organização por entidade.
- Rotas no padrão RESTful.
- Separação entre apresentação e negócio.
- Mais clareza nas requisições.

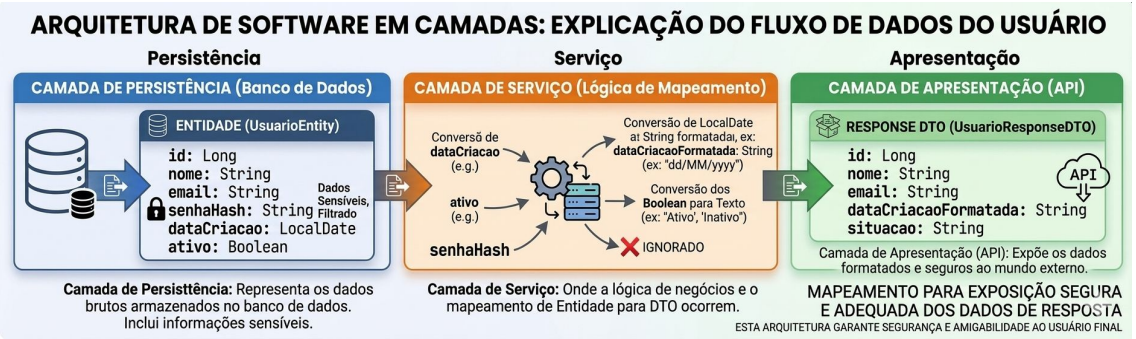
UsuarioController

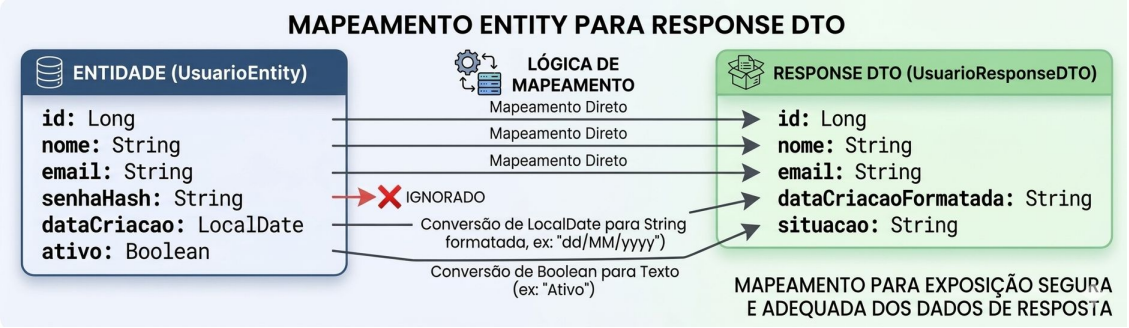
```
// API 04 - Visualizar Perfil
@GetMapping(path = "/{usuarioId}))
public ResponseEntity<?> visualizarPerfil(@PathVariable(parameter = "usuarioId" )
{   // Regra de negócio: buscar usuário pelo e-mail (identificador único)
    long idNumerico;
    try {
        idNumerico = Long.parseLong(usuarioId);
    } catch (NumberFormatExeption e) {
        return ResponseEntity.stusas(400, "ID inválido.");
    }
    // Regra de negócio: se não existir, retornar 404
    if (usuario == null) {
        return ResponseEntity.stusas(404, "Usuário não encontrado");
    }
    if (usuario.getDataNascimento() == null) {
        return ResponseEntity.stusas(400, "Usuário possui data de nascimento inválida.");
    }
}
```

Resultados

Mappers

- Conversão entre Model ↔ DTO
- Redução de acoplamento
- Código mais limpo e reutilizável
- Facilita manutenção da API





Considerações finais

Desafios Superados

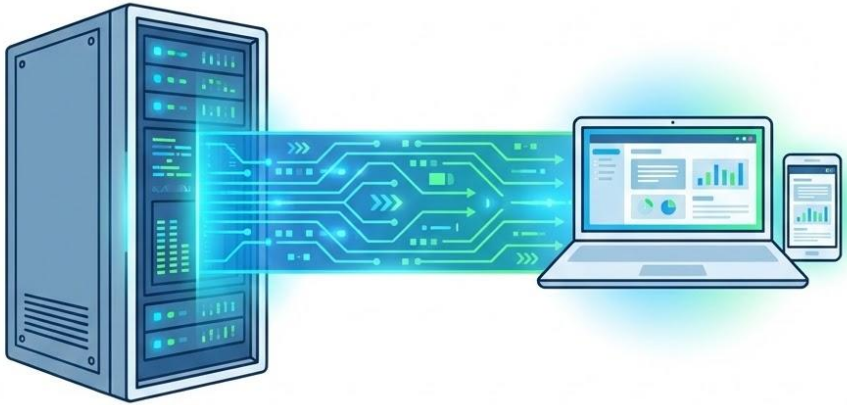
- Aplicação do Builder Pattern para objetos complexos
- Uso de DTOs para separar API e Model
- Organização em arquitetura em camadas
- Implementação de Mappers para conversão e baixo acoplamento

Considerações finais

Próximos Passos na Dendê Eventos

Persistência de dados em banco de dados

- Substituir o repositório em memória por um banco de dados real.
- Utilização de tecnologias como MySQL ou PostgreSQL para armazenamento permanente das informações.



Referências

- Curso de Java #01 - História do Java - Gustavo Guanabara. Disponível em: https://www.youtube.com/watch?v=sTX0UEplF54&list=PLHz_AreHm4dkI2ZdjTwZA4mPMxWTfNSpR. Acesso em: 21 fev. 2026.
- Curso de POO Java (Programação Orientada a Objetos). Disponível em: http://www.youtube.com/playlist?list=PLHz_AreHm4dkqe2aR0tQK74m8SFe-aGsY. Acesso em: 21 fev. 2026.
- ORACLE. Java Documentation. Disponível em: https://docs.oracle.com/en/java/. Acesso em: 21 fev. 2026.
- POSTMAN Blog – REST API Best Practices. [*S. l.*], 22 dez. 2025. Disponível em: https://blog.postman.com/rest-api-best-practices/. Acesso em: 21 fev. 2026.
- DEV Community – Ultimate Guide to HTTP Status Codes in REST APIs. [*S. l.*], 2 set. 2025. Disponível em: https://dev.to/gianfcop98/the-ultimate-guide-to-http-status-codes-in-rest-apis-40cp. Acesso em: 21 fev. 2026.

Centro Universitário de Excelência
Sistemas de Informação

Refatoração e Evolução da API DendêEventos

Autor: André Almeida Gomes Neto
Augusto Brandão Ferreira
Deivid Souza dos Santos Oliveira
Hellen Verena da Conceição Magalhães
Ismael Rodrigues de Oliveira Neto
João Vítor da Conceição de Almeida

Feira de Santana, 2026